

KUMARAGURU COLLEGE OF TECHNOLOGY

Department of Artificial Intelligence and Data Science

24ADT005-DEEP LEARNING

PROJECT REPORT

"BYAKU"

AI-Powered Spatial Audio Assistant for Low Vision Users

Real-time object detection, tracking, relative-depth estimation and spoken scene narration with human-in-the-loop review

Subject: Deep Learning

Submitted by

Navithanjali M S (24BAD077) ·

Piriyadharshini N (24BAD086) ·

Pradeep Manikandan D (24BAD088) ·

Sastha Jeyasri A (24BAD106) ·

Vishal S (24BAD130)

Faculty Guide: Ms.SriAarthy E

Academic Year: 2026-2027

Semester: V

Date of Submission: 30/10/2026

ABSTRACT

People who are blind or have low vision find it difficult to understand which objects are around them and where those objects are. Conventional object detectors report only labels and bounding boxes, are not designed for continuous video, and can be confidently wrong, so a user has no way to verify or correct them. This project, BYAKU, is a camera-based spatial audio assistant that turns visual scenes into short spoken descriptions such as “a bottle on the left, near”.

The system combines several deep learning components in a single pipeline. A YOLO11n convolutional object detector, fine-tuned on a 600-image, seven-class subset of Open Images V7 (420 training, 120 validation, 60 test images; 2,167 reviewed boxes) on top of COCO pretraining, detects objects. ByteTrack assigns persistent identities across frames. Depth Anything V2 (Small) estimates relative depth, which is grouped into near, mid-range and far zones, and bounding-box position gives direction (left, centre, right, top, bottom). A short-term scene memory, implemented with a NetworkX relationship graph, keeps temporarily hidden objects for up to five frames to reduce narration flicker. Narration is generated every five seconds and users may ask voice questions such as “Where is my bottle?”. A human-in-the-loop review dashboard stores audit-logged confirmations and corrections without retraining the model. The backend is built with FastAPI, with a browser front end served through a Node.js relay.

On the 60-image test split, the fine-tuned YOLO checkpoint is reported in the project presentation to reach 85.71% precision, 82.34% recall, 75.35% mAP@0.5 and 57.58% mAP@0.5:0.95 over 210 ground-truth boxes. These figures are taken from the presentation and have not been re-run for this report. Training hyperparameters, latency benchmarks and baseline comparisons are not available in the supplied material and are marked as such. BYAKU demonstrates the practical application of convolutional feature extraction, transfer learning and monocular depth estimation to an accessibility problem.

Keywords: deep learning, computer vision, YOLO11, object tracking, monocular depth estimation, assistive technology, human-in-the-loop.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1
1.1 Project Overview 1
1.2 Background 1
1.3 Problem Statement 1
1.4 Motivation 2
1.5 Project Objectives 2
Primary objectives (presentation, slide 4) 2
Secondary objectives 2
1.6 Scope of the Project 2
1.7 Significance of the Project 3
1.8 Organization of the Report 3
CHAPTER 2: PROBLEM ANALYSIS AND EXISTING SYSTEM 4
2.1 Detailed Problem Analysis 4
2.2 Existing System 4
2.3 Limitations of the Existing System 4
2.4 Proposed System 5
2.5 Advantages of the Proposed System 5
2.6 Functional Requirements 5
2.7 Non-Functional Requirements 5
2.8 System Constraints and Assumptions 6
CHAPTER 3: DEEP LEARNING CONCEPTS AND THEORETICAL BACKGROUND 7
3.1 Introduction to Deep Learning 7
3.2 Relevant Machine Learning and Deep Learning Concepts 7
3.3 Primary Deep Learning Model: YOLO11n 7
3.4 Mathematical Foundations 8
3.5 Why This Deep Learning Approach Was Selected 9
3.6 Role of Each Major Component 9
CHAPTER 4: COURSE OUTCOME MAPPING AND DEEP LEARNING CONCEPT MAPPING 10
4.1 Course Outcome Mapping 10
4.2 Primary Course Outcome 11
4.3 Deep Learning Concept Mapping 11
4.4 Problem-to-Solution-to-CO Relationship 12
4.5 Mapping to Deep Learning Syllabus Units 13
4.6 Final CO and Concept Summary 13
CHAPTER 5: SYSTEM ARCHITECTURE AND METHODOLOGY 14

5.1 Overall System Architecture 14
5.2 System Workflow 14
5.3 Dataset Description 15
5.4 Data Preprocessing 16
5.5 Model Architecture 16
5.6 Training Methodology 17
5.7 Prediction and Inference 17
5.8 Evaluation Methodology 18
5.9 Technology Stack 18
CHAPTER 6: IMPLEMENTATION DETAILS 20
6.1 Development Environment 20
6.2 Repository Structure 20
6.3 Module-Wise Implementation 20
6.4 Core Code Explanation 21
6.5 Execution Procedure 22
6.6 Implementation Challenges 22
CHAPTER 7: RESULTS AND PERFORMANCE ANALYSIS 23
7.1 Experimental Setup 23
7.2 Results 23
7.3 Performance Metrics 24
7.4 Training and Validation Analysis 24
7.5 Results Interpretation 24
7.6 Comparison with Baseline or Existing Approaches 24
7.7 Screenshots and Visualizations 25
7.8 Limitations of the Results 27
CHAPTER 8: APPLICATIONS, ADVANTAGES, AND LIMITATIONS 29
8.1 Real-World Applications 29
8.2 Advantages 29
8.3 Technical Limitations 29
8.4 Computational Requirements 30
8.5 Ethical, Privacy, and Security Considerations 30
CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS 31
9.1 Conclusion 31
9.2 Future Enhancements 31
APPENDIX A: IMPLEMENTATION EVIDENCE AND REPOSITORY STRUCTURE 34
A.1 Compact repository tree 34
A.2 Implementation evidence 34

APPENDIX B: MODEL CONFIGURATION (TO CONFIRM).....**Error! Bookmark not defined.**
APPENDIX C: COURSE OUTCOME EVIDENCE 35
APPENDIX D: RESULTS AND EVIDENCE CHECKLIST. **Error! Bookmark not defined.**

Note: Right-click the table above and choose "Update Field" (or press F9) in Microsoft Word to refresh entries and page numbers.

LIST OF FIGURES

No.	Title
Figure 5.1	Overall system architecture of BYAKU
Figure 5.2	Class distribution of the reviewed fine-tuning dataset
Figure 5.3	BYAKU landing / assistant page (from project presentation)
Figure 5.4	Camera preview and question box of the assistant page
Figure 7.1	Reported test-set metrics of the fine-tuned YOLO checkpoint
Figure 7.2	Detection, tracking and direction/depth labels on live-camera frames
Figure 7.3	Image-upload test: input image and annotated output with narration
Figure 7.4	Offline video analysis: upload interface and selected video
Figure 7.5	Offline video analysis: per-frame result cards for sampled frames
Figure 7.6	Human Review dashboard with per-object confirmation and correction

LIST OF TABLES

No.	Title
Table 3.1	Deep learning concepts used and their role
Table 4.1	Course Outcome mapping
Table 4.2	Deep learning concept mapping
Table 4.3	Problem-to-solution-to-CO relationship
Table 4.4	Syllabus unit mapping (to be completed)
Table 5.1	Dataset summary
Table 5.2	YOLO11n architecture overview
Table 5.3	Training configuration status
Table 5.4	Technology stack
Table 6.1	Repository structure
Table 6.2	Module-wise implementation
Table 7.1	Reported evaluation results
Table 7.2	Other reported validation
Table 7.3	Result-table templates for missing experiments
Table 9.1	Objective-wise achievement status
Table A.1	Implementation evidence
Table A.2	Items needed to finalise the report

Page numbers can be added after final formatting.

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
AP / mAP	Average Precision / mean Average Precision
API	Application Programming Interface
CC BY	Creative Commons Attribution licence
CNN	Convolutional Neural Network
CO	Course Outcome
COCO	Common Objects in Context (dataset)
CPU / GPU	Central / Graphics Processing Unit
DL	Deep Learning
FPS	Frames Per Second
HITL	Human-in-the-Loop
HTTP	Hypertext Transfer Protocol
IoU	Intersection over Union
JPEG	Joint Photographic Experts Group (image format)
JSON	JavaScript Object Notation
SHA-256	Secure Hash Algorithm, 256-bit
SiLU	Sigmoid Linear Unit
TTS	Text-to-Speech
UI	User Interface
ViT	Vision Transformer
WCAG	Web Content Accessibility Guidelines
WebSocket / WS	Bidirectional web communication protocol
YOLO	You Only Look Once

CHAPTER 1: INTRODUCTION

1.1 Project Overview

BYAKU is an AI-powered spatial audio assistant for people who are blind or have low vision. It uses a camera (live, uploaded image or uploaded video) to detect and track objects, estimates how far each object is relative to others and in which direction it lies, and converts this into short spoken sentences such as “A bottle on the left, near. A chair in the center, mid-range.” Users can also ask questions by voice, for example “Where is my bottle?”, and a human reviewer can verify or correct detections through a review dashboard.

Technically, the project is a computer-vision pipeline: a fine-tuned YOLO11n detector [1], the ByteTrack tracker [3], the Depth Anything V2 (Small) relative-depth model [4], rule-based spatial reasoning and scene memory, and a narration/query layer. It is exposed through a FastAPI backend and a browser front end served through a Node.js relay (README; presentation, slide 5).

1.2 Background

Understanding the surroundings is a basic requirement for independent living. Vision-impaired users depend on others or on tools that describe the scene. Modern deep learning object detectors such as YOLO [2] can recognise many everyday objects in real time, and monocular depth models such as Depth Anything V2 [4] can estimate scene structure from a single camera. Combining these with speech output makes it possible to give a person usable spatial information without requiring them to look at a screen.

1.3 Problem Statement

People who are blind or have low vision struggle to understand the objects and layout around them. A standard object detector names objects but does not say where they are; video contains changing objects and scenes that need continuous analysis; and AI predictions can be wrong, so users need a way to verify and correct them (presentation, slide 3).

Problem statement: To build a real-time system that detects and tracks objects, estimates relative distance and direction, narrates the scene in natural speech, and lets humans verify AI predictions.

- **Who is affected:** blind and low-vision users, and their caregivers/reviewers.
- **Why it matters:** spatial awareness supports safe, independent movement and finding everyday items.

- **Limitation of plain detection:** labels and boxes are visual outputs that carry no direction or distance, and they are unusable without a screen.
- **Why deep learning:** detecting objects across varied indoor scenes, lighting and viewpoints, and estimating depth from one image, require learned feature extraction; hand-written rules cannot do this reliably.

1.4 Motivation

The motivation is to convert raw detector output into information that a person can act on without seeing a screen, while keeping a human reviewer in control of AI errors. The distinctive aspects of the approach are the combination of tracking, relative depth and direction in one narration, an eyes-free voice-driven interface, and an audited review process that does not silently retrain the model.

1.5 Project Objectives

Primary objectives (presentation, slide 4)

1. Detect objects from the live camera, images and videos.
2. Track objects with persistent IDs across frames.
3. Classify relative depth as near, mid-range or far.
4. Identify direction: left, center, right, top, bottom.
5. Generate concise speech narration and answer voice questions.

Secondary objectives

1. Provide Human-in-the-Loop review to verify and correct detections.
2. Provide an accessible, eyes-free, voice-activated interface.

1.6 Scope of the Project

Implemented and documented (repository README/API contract): browser-based camera capture and image/video upload; YOLO11n detection with ByteTrack; optional Depth Anything V2 depth; spatial reasoning and scene answers; HTTP and WebSocket APIs; persistent human-review feedback store; and a pytest test suite in `Workflow/tests`.

Boundaries: depth is relative (no metre distances); the feedback store does not retrain the detector; the backend cannot confirm that browser speech was actually played; the detector is fine-tuned for seven assistive classes only; the project is not presented as a certified navigation or safety device. **Planned extensions** are listed in Chapter 9.

1.7 Significance of the Project

- **Academic:** applies CNN-based detection, transfer learning, tracking and monocular depth estimation within one working system.
- **Technical:** shows how separate deep learning models are integrated into a real-time service with a defined API, scene memory and auditing.
- **Practical:** targets an accessibility need by producing spatial speech rather than only visual overlays.

1.8 Organization of the Report

- Chapter 1 introduces the project. Chapter 2 analyses the problem, existing approaches, the proposed system and requirements.
- Chapter 3 presents the deep learning background. Chapter 4 maps the project to the Course Outcomes and concepts.
- Chapter 5 describes architecture and methodology. Chapter 6 describes the implementation. Chapter 7 presents available results.
- Chapter 8 covers applications, advantages and limitations. Chapter 9 concludes with future work. References and appendices follow.

CHAPTER 2: PROBLEM ANALYSIS AND EXISTING SYSTEM

2.1 Detailed Problem Analysis

Turning a camera image into useful spoken guidance involves several technical challenges:

- **Recognition:** everyday objects vary in size, pose, lighting and occlusion in indoor scenes.
- **Localisation in space:** a bounding box gives image position but not distance; direction and depth must be derived separately.
- **Temporal consistency:** in video, an object seen in consecutive frames must keep the same identity, and brief occlusions should not cause the object to "disappear" and be re-announced.
- **Latency:** frames must be processed quickly enough for continuous use; the presentation reports about 100 ms per frame for detection (hardware not stated).
- **Communication:** output must be concise speech, not a stream of every detection.
- **Reliability:** predictions can be wrong even at high confidence, so correction and audit are needed.

2.2 Existing System

Conventional computer-vision approaches use a single-frame object detector (such as YOLO [2]) that outputs class labels, confidence and bounding boxes, usually displayed visually. Tracking-by-detection methods such as ByteTrack [3] add identities across frames, and monocular depth networks [4] estimate relative scene depth. These are separate components in the literature; the presentation does not name any competing commercial assistive product, so none is compared in this report.

2.3 Limitations of the Existing System

- A plain detector names objects but does not say where they are (presentation, slide 3).
- Frame-by-frame detection has no memory, so the same object is treated as new in each frame.
- Visual overlays are unusable for the target users without speech output.
- Predictions cannot be verified or corrected by the user or a helper in a plain detector.

2.4 Proposed System

BYAKU addresses these limitations by chaining detection, tracking, relative depth, direction reasoning, scene memory and narration, and by adding a human review process. Live-camera sessions and offline uploads are processed through separate workflows (presentation, slides 8 and 18).

2.5 Advantages of the Proposed System

- Gives direction and relative distance together with the object name.
- Persistent IDs and a five-frame memory reduce flickering narration.
- Eyes-free interaction: voice "START/STOP", voice questions, screen-reader support.
- Audited human correction without automatic retraining, which avoids feeding unreviewed feedback into the model.

> *No claim is made that BYAKU is more accurate, faster or cheaper than other systems, because no comparative experiment is available.*

2.6 Functional Requirements

1. Accept frames from the browser camera, an uploaded image or an uploaded video.
2. Decode and validate frames; process one frame at a time per session.
3. Detect objects and return class, confidence and bounding box; assign persistent track IDs.
4. Estimate relative depth and classify objects as near, mid-range or far.
5. Determine direction (left, center, right, top, bottom).
6. Generate narration about every five seconds and answer user queries from the current scene memory.
7. Provide health, status, frame, query, camera and feedback endpoints and a WebSocket channel.
8. Allow human confirmation/correction of detections with undo and audit history.

2.7 Non-Functional Requirements

Requirement	Description / evidence
Performance	Near real-time use; detection about 100 ms/frame is reported (hardware not stated, not independently verified).
Reliability	Single in-flight frame; structured error messages; HTTP 409 for overlapping operations (API contract).

Requirement	Description / evidence
Usability / accessibility	High-contrast night mode (WCAG AAA claimed), 48×48 px touch targets, aria-live regions, voice control (presentation, slide 26).
Maintainability	Separate Backend, Workflow, UI, Models, Data and Docs folders; automated tests reported (UI 38; backend 411 passed, 1 skipped).
Security / privacy	Camera starts only on user action; frames are not written to review history; correction tokens expire after 180 s.
Scalability	Not evaluated; the design is a single local backend instance.
Resource needs	Python environment with PyTorch/Ultralytics, Node.js, browser camera and microphone; GPU requirement not stated.

2.8 System Constraints and Assumptions

- The launcher does not install dependencies or download models; the Python environment (`.venv-1`) and Node.js must already exist (README).
- Depth Anything V2 is used only when its model and dependencies are available locally.
- Depth is relative; no metre distances are produced.
- Server-side text-to-speech is reported unavailable; narration relies on browser speech synthesis and playback cannot be confirmed by the backend.
- Only sampled frames of an uploaded video are analysed; partial results are possible.

CHAPTER 3: DEEP LEARNING CONCEPTS AND THEORETICAL BACKGROUND

3.1 Introduction to Deep Learning

Deep learning uses multi-layer neural networks to learn hierarchical feature representations directly from data [8]. For images, early layers learn edges and textures and deeper layers learn object parts and objects. This removes the need for hand-designed features and is why deep networks dominate object detection and depth estimation, the two tasks at the core of BYAKU.

3.2 Relevant Machine Learning and Deep Learning Concepts

Table 3.1 lists the concepts relevant to this project. Concepts such as recurrent networks, GANs and autoencoders are not part of the verified implementation and are therefore not discussed.

Table 3.1: Deep learning concepts used and their role

Concept	Role in BYAKU	Status
Convolutional Neural Network (CNN)	Backbone/neck/head of YOLO11n for feature extraction and detection	Used (presentation; README)
Object detection (one-stage)	Predict boxes, classes and confidence in one forward pass	Used
Transfer learning / fine-tuning	COCO-pretrained YOLO11n fine-tuned on 7 assistive classes (Open Images V7 subset)	Dataset and test metrics reported; training settings not provided
Loss functions, backpropagation, optimisation	Used when fine-tuning the detector	Framework-level; exact settings to be confirmed
Multi-object tracking (tracking-by-detection)	ByteTrack assigns persistent IDs; association is not a trained network in this project	Used
Monocular depth estimation	Depth Anything V2 Small gives relative depth for near/mid/far zones	Used when available locally
Evaluation metrics	Precision, recall, mAP@0.5, mAP@0.5:0.95	Reported
Human-in-the-loop learning data curation	Reviewed corrections stored for possible dataset curation; no automatic retraining	Implemented as storage only

3.3 Primary Deep Learning Model: YOLO11n

Purpose. YOLO11n is the smallest model of the Ultralytics YOLO11 family and is the main model in the pipeline. It detects the seven assistive classes in each frame.

Input representation. Each decoded frame is resized to a fixed inference size of 640 (imgsz=640, slide 11) and normalised into a tensor.

Layer structure. YOLO-family detectors consist of a convolutional backbone that extracts multi-scale features, a neck that fuses them, and a detection head that outputs, for every spatial location, box coordinates and class scores [1], [2]. The block-level layout is given in Section 5.5.

Output. A set of boxes with class label and confidence, which are filtered by confidence and non-maximum suppression (framework behaviour) before being passed to ByteTrack.

Forward propagation, loss and backpropagation. During fine-tuning the network predicts boxes and classes for labelled images; a loss compares predictions to ground truth (a box-regression term and a classification term); gradients are obtained by backpropagation and weights are updated by an optimiser. The exact loss weights, optimiser and learning-rate schedule used in the project run are **Not provided** and must be confirmed from the training logs.

Inference. At runtime only the forward pass is executed on each validated frame; the project reports about 100 ms per frame for detection (hardware not stated).

3.4 Mathematical Foundations

Neuron computation. Each unit computes a weighted sum followed by a non-linearity:

$$y = f(\Sigma_i\, w_i x_i + b)$$

where x_i are inputs, w_i weights, b the bias and f the activation function.

2-D convolution (as used in the convolutional layers):

$$(I * K)(i, j) = \Sigma_m\, \Sigma_n\, I(i+m, j+n) \cdot K(m, n)$$

where I is the input feature map and K the learnable kernel.

Activation. Ultralytics YOLO models use the SiLU activation in convolution blocks (framework design; to be confirmed against the model file) and sigmoid for class probabilities:

$$\mathrm{SiLU}(x) = x \cdot \sigma(x), \quad \sigma(x) = 1 \,/\, (1 + e^{-x})$$

Binary cross-entropy for classification, with label y and predicted probability p:

$$L_cls = -[\, y \cdot \log(p) + (1 - y) \cdot \log(1 - p) \,]$$

Intersection over Union, used for box quality and in detection loss and evaluation:

$$IoU = |A \cap B| \,/\, |A \cup B|$$

where A is the predicted box and B the ground-truth box.

Gradient-descent update used by the optimiser:

$$\theta \leftarrow \theta - \eta \cdot \nabla\theta \, L$$

where θ are the parameters, η the learning rate and L the loss.

Evaluation metrics. With TP, FP, FN denoting true positives, false positives and false negatives:

$$\text{Precision} = TP / (TP + FP), \quad \text{Recall} = TP / (TP + FN)$$

$$AP = \int_0^1 p(r)\, dr, \quad mAP = (1/N) \, \Sigma \, AP_k \text{ over N classes}$$

mAP@0.5 uses an IoU threshold of 0.5; mAP@0.5:0.95 averages over IoU thresholds from 0.5 to 0.95 and is stricter.

Tracking-by-detection. ByteTrack [3] associates detections to existing tracks using motion prediction and overlap, and it also uses low-confidence detections for association, which helps keep identities during brief occlusion. This stage is not a network that is trained in this project.

Relative depth. Depth Anything V2 predicts a per-pixel relative depth map from one RGB image [4]. BYAKU normalises it and groups objects into three zones (near, mid-range, far); no metric distance is claimed.

3.5 Why This Deep Learning Approach Was Selected

- **YOLO11n** is a small one-stage detector suitable for near real-time use on modest hardware, which matches the live-camera requirement. Larger detectors or two-stage detectors were not tested in this project, so no experimental comparison is claimed.
- **Transfer learning** from COCO allows useful detection with a small fine-tuning set (600 images) instead of training from scratch.
- **ByteTrack** adds identities without training another network.
- **Depth Anything V2 Small** provides depth from a single ordinary camera, avoiding special depth sensors.

3.6 Role of Each Major Component

The dataset (Open Images subset, reviewed boxes) provides labels for fine-tuning and evaluation. Preprocessing validates frames and resizes them for the detector. YOLO11n produces detections; ByteTrack adds IDs; Depth Anything V2 adds relative depth; spatial reasoning converts boxes and depth to direction and zone; scene memory stabilises results; narration and query logic produce sentences; the metrics (precision, recall, mAP) assess the detector; and the Human Review dashboard records corrections for traceability.

CHAPTER 4: COURSE OUTCOME MAPPING AND DEEP LEARNING CONCEPT MAPPING

4.1 Course Outcome Mapping

The official Course Outcomes were supplied as an image of the syllabus table and are quoted exactly. The mapping below is based on the presentation, README and API documentation. Source code was not available for inspection, so the evidence column refers to documented components and to files named in the repository.

Table 4.1: Course Outcome mapping

CO	Official Course Outcome	Relevant project component	Evidence from the project	Justification and strength
CO2	Implement deep learning techniques for feature extraction, image analysis, and computer vision applications.	YOLO11n detection, ByteTrack tracking, Depth Anything V2 depth, spatial reasoning	Presentation slides 11, 12, 15; README; `Workflow/` detection and depth adapters; test-set metrics (slide 27)	**Direct (strongest).** A complete computer-vision application with detection, tracking and depth on image, video and live-camera input.
CO1	Apply neural network architectures, activation functions, optimization methods, and regularization techniques to develop and evaluate deep learning models for predictive tasks.	Fine-tuning and evaluation of the YOLO11n detector	Open Images subset (420/120/60), evaluation on 60 images with P, R, mAP (slides 7, 27); `Models/` checkpoints; `Workflow/` training tools	**Partial.** Fine-tuning and evaluation are evidenced, but training hyperparameters, optimiser, regularisation and curves are not provided, so the "apply optimisation and regularisation" part cannot be demonstrated yet.
CO5	Analyze attention-based architectures and advanced deep learning techniques to develop intelligent applications for language and multimodal AI tasks.	Depth Anything V2 (transformer-based, per [4]); voice/text query answering	Slides 15–16; `/api/query` and WebSocket `query` message (API contract)	**Indirect / weak.** The depth model is transformer-based and the app links vision with spoken language, but narration and query answering appear rule-based, and no attention analysis is provided.
CO3	Develop sequential learning models for processing and analyzing sequential and natural language data.	—	No recurrent/sequence model verified	**Not supported.** Frame sequences are handled by tracking and scene memory, not by a learned sequential model.

CO	Official Course Outcome	Relevant project component	Evidence from the project	Justification and strength
CO4	Analyze generative and unsupervised deep learning techniques for representation learning, feature extraction, and synthetic data generation.	—	No generative or unsupervised model verified	**Not supported.**

4.2 Primary Course Outcome

CO2 (RBT level: Ap) is the strongest match. The project's central activity is the implementation of deep learning techniques for feature extraction and image analysis in a computer-vision application: CNN-based detection of seven object classes, multi-object tracking across frames, and monocular depth estimation on live-camera, image and video inputs. The practical skill demonstrated is integrating pretrained and fine-tuned vision models into an end-to-end pipeline with an API. Evidence of learning is the working pipeline described in the repository and the evaluation of the fine-tuned detector on a held-out 60-image test split.

CO1 is a secondary alignment that becomes direct once the training configuration and logs are added (see Appendix B).

4.3 Deep Learning Concept Mapping

Table 4.2: Deep learning concept mapping

Concept	How it works	Where used	Relevant module	Contribution
CNN-based one-stage object detection (YOLO11n)	Convolutional layers extract multi-scale features; the head predicts boxes and classes in one pass	Step 2 of the pipeline	`Models/yolo11n.pt` (active checkpoint); detector adapter in `Workflow/`	Finds objects and confidence
Transfer learning / fine-tuning	Start from COCO weights and continue training on new classes	Open Images V7 subset, 7 classes	Training tools in `Workflow/`; checkpoints in `Models/`	Adapts detector to assistive classes with few images
Tracking-by-detection	Associates detections across frames using motion and overlap	Step 3	ByteTrack in the detection adapter	Persistent IDs; handles brief occlusion

Concept	How it works	Where used	Relevant module	Contribution
Monocular depth estimation	Network predicts relative depth per pixel from one RGB image	Step 4	Depth adapter in `Workflow/` (optional)	Near/mid/far zones
Evaluation metrics	Precision, recall, AP/mAP computed from IoU matches	Testing on 60 images	Evaluation output reported on slide 27	Quantifies detector quality
Human-in-the-loop	Human confirms or corrects model output; corrections stored with audit trail	Review dashboard	`Backend/feedback_store.py`	Verifiable, traceable predictions; no auto-retraining

Connection between theory and implementation: the convolution and loss equations of Chapter 3 correspond to the YOLO detector in `Models/`, the IoU and precision/recall/mAP formulas correspond to the evaluation figures, and the depth map is turned into near/mid/far groups by the spatial reasoning module.

4.4 Problem-to-Solution-to-CO Relationship

Table 4.3: Problem-to-solution-to-CO relationship

Identified problem	Proposed solution	DL technique	Implemented component	CO	Evidence
Objects are not recognised from camera input	Real-time detection	CNN detector (YOLO11n)	Detector adapter, `Models/yolo11n.pt`	CO2, CO1	Slide 27 metrics; slide 12
Same object is treated as new in each frame	Persistent IDs	ByteTrack	Tracking stage	CO2	Slide 11; slide 12 labels “NEW”
Detection has no distance information	Relative depth zones	Depth Anything V2 Small	Depth adapter	CO2 (CO5 indirect)	Slide 15
Output is visual only	Spoken narration and voice queries	Rule-based reasoning over DL outputs	Scene reasoning; `/api/query`	CO5 (weak)	Slide 16; API contract
AI predictions can be wrong	Human review with audit log	Human-in-the-loop (no retraining)	`feedback_store.py`; dashboard	CO1 (evaluation mindset)	Slides 22–24; API contract

4.5 Mapping to Deep Learning Syllabus Units

Only the Course Outcomes were supplied; the official unit numbers and topic names of the Deep Learning syllabus were **not provided**. They are therefore not invented. The table below is a template to be completed once the syllabus is available.

Table 4.4: Syllabus unit mapping (to be completed)

Syllabus unit	Official topic	Relevant project concept	Implementation evidence	Explanation
[Not provided]	[Not provided]	CNN-based detection; transfer learning	YOLO11n fine-tuning (Chapter 5)	To be completed
[Not provided]	[Not provided]	Monocular depth (transformer-based encoder)	Depth Anything V2 stage	To be completed
[Not provided]	[Not provided]	Evaluation metrics	Precision, recall, mAP (Chapter 7)	To be completed

4.6 Final CO and Concept Summary

- **Problem solved:** blind and low-vision users cannot easily learn what is around them and where; BYAKU converts camera input into spoken spatial descriptions.
- **Primary model:** fine-tuned YOLO11n (CNN one-stage detector) with ByteTrack tracking and Depth Anything V2 Small relative depth.
- **Concepts demonstrated:** convolutional feature extraction, transfer learning, tracking-by-detection, monocular depth estimation, detection metrics and human-in-the-loop verification.
- **Most relevant CO:** CO2; CO1 partially, pending training evidence.
- **Evidence:** documented pipeline and APIs in the repository, dataset split, and the reported test metrics.

CHAPTER 5: SYSTEM ARCHITECTURE AND METHODOLOGY

5.1 Overall System Architecture

BYAKU has four layers (Figure 5.1). The **browser client** captures the camera or uploads, handles voice START/STOP and speaks the output. The **Node.js relay** (port 3000) serves the static front end from `UI/` and proxies `/api/*`, `/health` and `/ws` to the **FastAPI backend** (port 8765). The backend decodes frames and runs the vision pipeline: YOLO11n detection, ByteTrack tracking, Depth Anything V2 relative depth, spatial reasoning, scene memory and narration. A **human-review store** records confirmations and corrections.

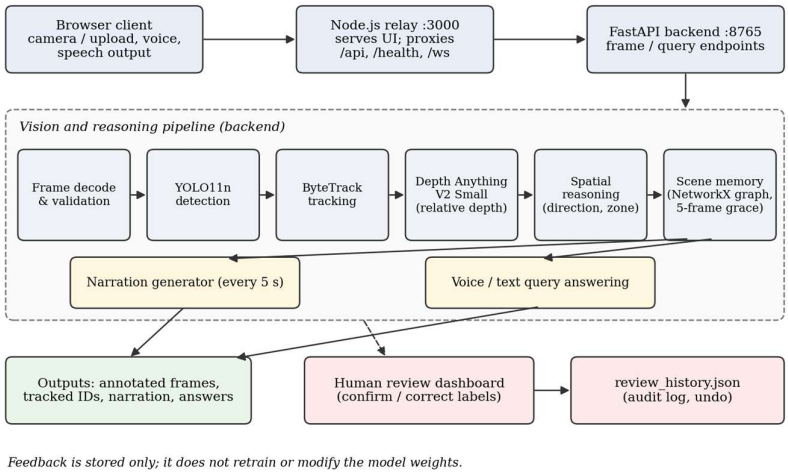


Figure 5.1: Overall system architecture of BYAKU

The diagram follows the presentation (slide 5) and the API contract. Depth Anything V2 is optional at run time: the README states it is used only when its model and dependencies are available locally.

5.2 System Workflow

1. **Input:** live camera through browser getUserMedia after permission (started by the voice command “START”), an uploaded image, or an uploaded video from which frames are sampled.
2. **Ingestion:** each frame is decoded and validated. Only one frame is in flight at a time; the next is sent after the previous is processed. Offline uploads are kept separate from the live session.
3. **Detection:** YOLO11n at imgsz=640 returns boxes, classes and confidence.

4. **Tracking:** ByteTrack assigns IDs (for a still image there is no cross-frame tracking).
5. **Depth and direction:** Depth Anything V2 Small gives normalised relative depth grouped into near/mid-range/far; box position gives left/center/right and top/bottom.
6. **Scene memory:** objects that disappear are kept as temporarily missing for up to five frames; a NetworkX relationship graph stores object relations.
7. **Narration and queries:** concise speech is produced about every five seconds; a voice question such as "Where is my bottle?" interrupts the routine narration and is answered from scene memory.
8. **Human review:** a frame token valid for 180 s allows review; corrections are stored with an audit trail.

5.3 Dataset Description

Table 5.1: Dataset summary

Item	Details (source: presentation, slide 7)
Fine-tuning dataset	Open Images V7 subset
Images	600 in total: 420 train, 120 validation, 60 test
Annotations	2,167 reviewed bounding boxes
Classes (7)	book 570, person 519, chair 337, bottle 245, cup 198, laptop 151, cell phone 147
Data quality	Perceptual-hash audit removes cross-split duplicates; CC BY 2.0 licence verified (as stated)
Pretraining data	COCO / COCO128 (80 classes), the pretraining base of YOLO11n
Local samples	Indoor scenes and test videos used for warm-up and offline pipeline tests
Per-split box counts	Only the test split count (210 boxes) is stated; train/validation box counts: Not provided
Annotation format	Not stated (YOLO-format labels are typical; requires confirmation)

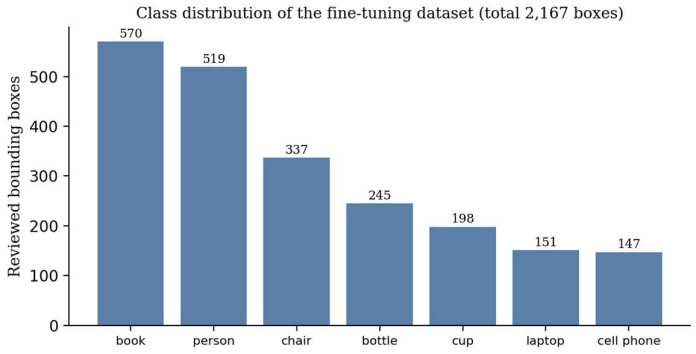


Figure 5.2: Class distribution of the reviewed fine-tuning dataset

The distribution is imbalanced: “book” has about 3.9 times as many boxes as “cell phone”. This may affect per-class performance (see Section 7.8). The test split is small (60 images), so metrics should be interpreted with caution.

5.4 Data Preprocessing

- **Frame preprocessing at inference:** decode base64 JPEG, validate positive width/height and frame identity, resize to imgsz=640 for the detector.
- **Dataset preparation (from presentation):** manual review of boxes (2,167 reviewed boxes), perceptual-hash duplicate removal across splits, and licence verification.
- **Depth post-processing:** depth values are normalised and grouped into three zones.
- **Data augmentation:** Not provided. Ultralytics applies default augmentations during training unless disabled; whether the project changed them requires confirmation.

5.5 Model Architecture

The active detector is YOLO11n (`Models/yolo11n.pt`, or the locally trained checkpoint configured in code). The layer-level description below is the standard YOLO11 design from the Ultralytics documentation [1]; exact layer dimensions and parameter count were not verified from the project files and should be confirmed with `model.info()` on the project checkpoint.

Table 5.2: YOLO11n architecture overview

Stage	Typical blocks (Ultralytics YOLO11)	Function
Input	640 × 640 × 3 image tensor (imgsz=640)	Fixed-size input
Backbone	Strided Conv layers, C3k2 blocks, SPPF, C2PSA (spatial attention block)	Multi-scale feature extraction
Neck	Upsampling and concatenation of feature maps, C3k2 blocks (PAN/FPN-style)	Fuse features from several scales
Head	Decoupled detection head on three scales	Box coordinates and class scores
Output	Boxes, class, confidence (7 classes after fine-tuning)	Passed to ByteTrack
Activation	SiLU in conv blocks; sigmoid class scores	Non-linearity
Parameters	Not verified for the project checkpoint	Confirm with model summary

Depth Anything V2 Small uses a vision-transformer encoder with a depth decoder to output a relative depth map [4]. Its internal layers are not modified by the project; it is used for inference only.

5.6 Training Methodology

Fine-tuning of YOLO11n on the Open Images subset is described in the presentation, and `Workflow/` is described in the README as containing training tools, with "preserved custom-training checkpoints/runs" in `Models/`. The training configuration itself was not available. Table 5.3 separates what is known from what must be confirmed.

Table 5.3: Training configuration status

Parameter	Value	Status
Base model	YOLO11n pretrained on COCO	Reported (slide 7)
Dataset split	420 / 120 / 60 images	Reported
Input size	640 (inference); training size not stated	Partly reported
Epochs	—	Not provided
Batch size	—	Not provided
Optimizer / learning rate	—	Not provided (Ultralytics selects automatically by default; confirm)
Loss function	Detection loss (box + classification terms), framework default	Requires confirmation
Regularisation / augmentation	—	Not provided
Checkpointing / early stopping	Checkpoints preserved in `Models/`	Details not provided
Hardware	—	Not provided

Validation strategy: the 120-image validation split is used during training and the 60-image test split for the final evaluation (reported on slide 27).

5.7 Prediction and Inference

For a new frame, the browser sends a JPEG (base64) with width, height and frame identity to `POST /api/frame` or the WebSocket `frame` message. The backend runs the pipeline and returns the result and server-side timing (`initialization_ms`, `processing_ms`, `timing_source`). Queries are sent through `POST /api/query` or a WebSocket `query` message and are answered from the latest committed scene. The front end shows boxes with labels (e.g., "PERSON 77% | NEW | CENTER | FAR") and reads the narration with browser speech synthesis.

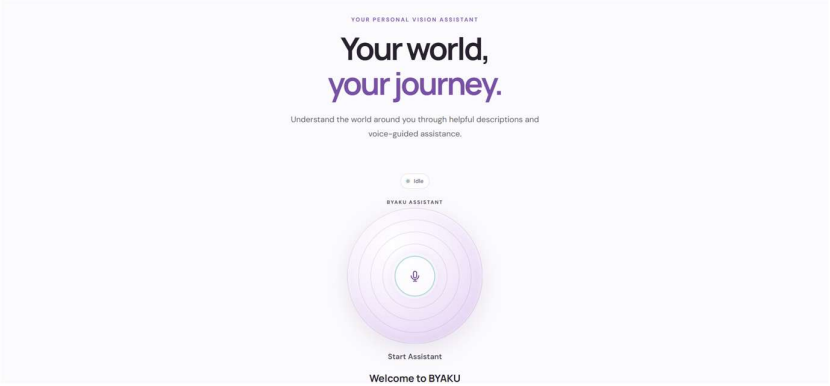


Figure 5.3: BYAKU landing / assistant page (from project presentation)

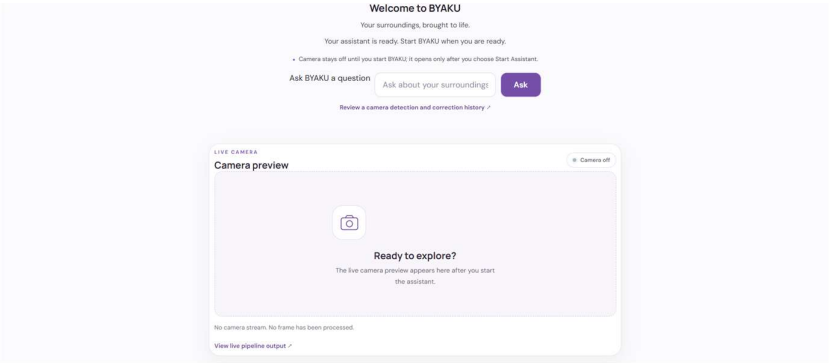


Figure 5.4: Camera preview and question box of the assistant page

5.8 Evaluation Methodology

The detector is evaluated on the held-out test split with precision, recall, mAP@0.5 and mAP@0.5:0.95. Precision shows how many predicted boxes are correct (important because wrong announcements mislead the user); recall shows how many real objects are found (important because missed objects are not announced); mAP summarises ranking quality across confidence thresholds and classes. The system is also tested through automated tests and manual image/video uploads (Chapter 7).

5.9 Technology Stack

Table 5.4: Technology stack

Technology	Version	Purpose	Where used
Python	Not stated	Backend language	`Backend/`, `Workflow/`
FastAPI / Uvicorn	>=0.110.0 / >=0.27.0	HTTP and WebSocket server	`Backend/fastapi_application.py`
Pydantic, websockets	>=2.6.0, >=12.0	Request validation, WebSocket support	Backend API
OpenCV, NumPy	>=4.10.0, >=1.26.0	Frame handling, array operations	Pipeline

Technology	Version	Purpose	Where used
PyTorch	>=2.2.0	Deep learning runtime	YOLO and depth inference
Ultralytics	>=8.2.0	YOLO11n inference and training	Detector, ByteTrack
NetworkX	>=3.2.0	Relationship graph (per presentation)	Scene memory
Depth Anything V2	Small; optional	Relative depth	Depth adapter
Node.js	Not stated	Relay and static front end	`UI/server/http_websocket_relay.js`
pytest, httpx	>=8.0.0, >=0.27.0	Automated tests	`Workflow/tests`
Browser APIs	—	getUserMedia camera; speech synthesis; aria-live	`UI/`

> *requirements.txt also lists gradio, pyttsx3, SpeechRecognition, PyAudio and vosk under "recommended project stack for later stages", and depth-anything-v2 and boxmot as optional. Their presence does not prove that they are used in the final workflow, so they are not claimed as implemented.*

CHAPTER 6: IMPLEMENTATION DETAILS

6.1 Development Environment

The README gives PowerShell commands and a Windows-style path to a local virtual environment (`.venv-1`), which indicates development on Windows. Python version, operating-system version, CPU/GPU and RAM are **Not provided**. The launcher expects Node.js and the `UI/` relay dependencies to be installed already.

6.2 Repository Structure

Table 6.1: Repository structure (from README)

Path	Contents
`UI/`	Browser front end, static pages (landing, `assistant.html`, `dashboard.html`), shared assets and Node relay
`Backend/`	`fastapi_application.py`, `browser_message_adapter.py`, `feedback_store.py`, `frame_timing_profiler.py`, `live_camera_producer.py`, `run_byaku.py`, `__init__.py`
`Workflow/`	Detection/depth adapters, spatial reasoning, scene pipeline, dataset utilities, training tools and tests (`Workflow/tests`)
`Models/`	Active `yolo11n.pt` checkpoint and preserved custom-training checkpoints/runs
`Data/`	Datasets, annotations, manifests, sample media, provenance documents, local review history
`Docs/`	Architecture/runtime notes and `API_AND_WEBSOCKET_CONTRACT.md`
`Config/`	Local Ultralytics configuration
`test videos/`	Sample videos for offline analysis
`requirements.txt`, `README.md`, `pyrightconfig.json`	Dependencies, documentation, type-checker configuration

The main entry point is `Backend/run_byaku.py`.

6.3 Module-Wise Implementation

Table 6.2: Module-wise implementation

Module	Purpose and processing	Inputs → outputs
`run_byaku.py`	Launcher: starts FastAPI on 127.0.0.1:8765 and the Node relay on 127.0.0.1:3000; bounded initial-frame timeout configurable by `BYAKU_INITIAL_FRAME_TIMEOUT_SECONDS` (1–300 s, default 180)	Environment → running services

Module	Purpose and processing	Inputs → outputs
`fastapi_application.py`	HTTP and WebSocket routes: `/health`, `/api/status`, `/api/frame`, `/api/query`, `/api/camera/*`, `/api/feedback`	Requests → JSON responses
`browser_message_adapter.py`	Handles browser WebSocket messages (ping, start/stop camera, frame, query) and structured errors	WS messages → pipeline calls
`live_camera_producer.py`	Optional Python camera producer controlled by `/api/camera/*`; browser camera is separate	Camera → frames
`frame_timing_profiler.py`	Timing support: initialization and per-frame processing times	Pipeline → timing values
`feedback_store.py`	Persistent human-review store (`Data/feedback/review_history.json`); undo preserves audit history	Review actions → JSON history
`Workflow/` pipeline	Detection (YOLO11n + ByteTrack), depth, spatial reasoning, scene memory, narration	Frame → scene description
`UI/`	Camera control, voice START/STOP, overlay, narration display, dashboard and Human Review	User → messages; results → speech/visuals

Module responsibilities are taken from the file names and the README/API contract. Class and function names inside `Workflow/` were not available and are therefore not quoted.

6.4 Core Code Explanation

Source files could not be read for this report, so line-level code explanation is **Not verifiable from the available files**. The documented message formats are shown below (from `Docs/API_AND_WEBSOCKET_CONTRACT.md`):

```
{"type":"ping","ts":42}                        -> pong
{"type":"frame","frame_id":1,"width":640,
 "height":480,"data":"<base64 JPEG>"}              -> frame result
{"type":"query","text":"What is to the left of the bed?"}
```

Behaviour specified by the contract: the frame result contains pipeline status and server-side timing; `/api/query` answers from the latest committed scene; feedback entries store the frame reference, tracking ID, timestamp, original prediction, corrected value, correction type and review status; an exact normalised-crop match can apply a remembered correction to that one inference only, with the model's original class kept as `model_class_name`. **To complete this section, add short excerpts of the detector adapter, the spatial-reasoning function and the narration generator from `Workflow/`.**

6.5 Execution Procedure

1. **Prerequisites:** the existing `.venv-1` Python environment with the packages in `requirements.txt`, a local Node.js installation, and installed `UI/` relay dependencies. The launcher does not install dependencies or download models.
2. **Model:** `Models/yolo11n.pt` (or the configured local trained checkpoint) must be present. Depth Anything V2 is optional.
3. **Start (PowerShell, repository root):**

```
& ".venv-1\Scripts\python.exe" "Backend\run_byaku.py"
```

1. **Open:** `http://127.0.0.1:3000/` (landing), `/assistant.html` (live assistant), `/dashboard.html` (workflow status and Human Review). Camera stays off until "Start assistant" is chosen.
2. **Tests:**

```
& ".venv-1\Scripts\python.exe" -m pytest "Workflow\tests" -q -p no:cacheprovider
```

Training commands and dataset-preparation commands are **Not provided** and are not invented. Expected output: annotated frames, narration text and answers; stop the launcher with Ctrl+C.

6.6 Implementation Challenges

Only challenges evidenced in the documentation are listed:

- Frames must be processed one at a time, so the design allows a single in-flight frame and returns HTTP 409 for overlapping operations.
- Uploaded videos are sampled; only successfully analysed frames are shown and the result may be marked partial.
- The backend cannot confirm that browser speech was played; server-side TTS is reported unavailable.
- Depth depends on local model availability, so the pipeline must work without it.
- Feedback must not retrain the detector, to avoid data poisoning (presentation, slide 22).

CHAPTER 7: RESULTS AND PERFORMANCE ANALYSIS

> *All numerical results in this chapter are taken from the project presentation (slide 27). They have not been re-run or independently verified for this report. Items that were not supplied are marked "Not provided".*

7.1 Experimental Setup

- **Model evaluated:** the fine-tuned YOLO checkpoint (YOLO11n base).
- **Test data:** 60 held-out images with 210 ground-truth boxes across 7 classes.
- **Dataset split:** 420 train / 120 validation / 60 test, with cross-split duplicates removed by perceptual hashing.
- **Hardware, software versions, confidence/NMS thresholds:** Not provided.

7.2 Results

Table 7.1: Reported evaluation results (YOLO checkpoint, test split)

Metric	Value	Source
Precision	85.71%	Slide 27
Recall	82.34%	Slide 27
mAP@0.5	75.35%	Slide 27
mAP@0.5:0.95	57.58%	Slide 27

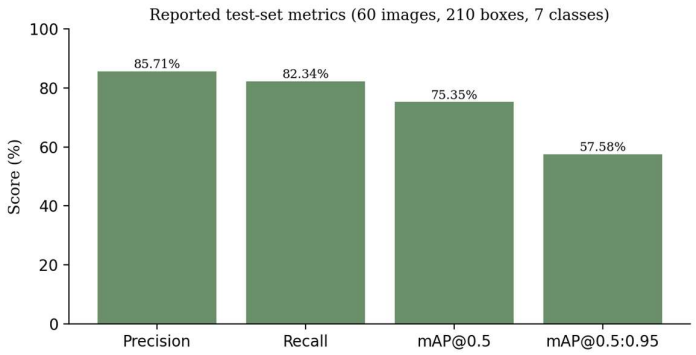


Figure 7.1: Reported test-set metrics of the fine-tuned YOLO checkpoint

Table 7.2: Other reported validation

Item	Reported outcome	Status
UI tests	38 passing	Previously reported (slide 27); not re-run
Backend tests	411 passing, 1 skipped	Previously reported; not re-run
Local image upload	Returned annotated output and a scene description	Reported; see Figure 7.3

Item	Reported outcome	Status
Packaged video upload	Processed sampled frames with a partial-result status	Reported; see Figures 7.4–7.5

7.3 Performance Metrics

Detection is a classification-and-localisation task, so precision, recall and mAP are appropriate (Section 3.4). Regression metrics such as MAE or segmentation metrics such as Dice are not relevant and are not reported. Depth quality is not measured because only relative depth zones are used and no depth ground truth is supplied.

7.4 Training and Validation Analysis

Training and validation loss curves, per-epoch metrics and logs were **not provided**, so convergence, overfitting and generalisation cannot be analysed here. The only generalisation evidence is the held-out test result in Table 7.1. If the `Models/` run folders contain `results.csv` and plots, they should be added as an appendix.

7.5 Results Interpretation

- Precision (85.71%) above recall (82.34%) indicates that the detector tends to be slightly more conservative: when it announces an object it is usually right, though some objects are missed.
- The gap between mAP@0.5 (75.35%) and mAP@0.5:0.95 (57.58%) is expected, since the stricter IoU thresholds penalise imprecise box placement. For BYAKU, class and approximate direction matter more than tight boxes, so mAP@0.5 is the more relevant indicator.
- These interpretations describe the reported numbers only; no per-class or error analysis is available.

7.6 Comparison with Baseline or Existing Approaches

No baseline result (for example, the COCO-pretrained YOLO11n evaluated on the same 60 images) was supplied, and no published result is directly comparable because the test set is project-specific. Therefore no comparison is made. Table 7.3 gives a template for the experiments that would enable it.

Table 7.3: Result-table templates for missing experiments (no values measured)

Experiment	Precision	Recall	mAP@0.5	mAP@0.5:0.95	Status
COCO-pretrained YOLO11n on the 7 classes (baseline)	—	—	—	—	Not run
Fine-tuned YOLO11n (this project)	85.71%	82.34%	75.35%	57.58%	Reported
Per-class results (7 classes)	—	—	—	—	Not provided
Latency per frame on stated hardware	—	—	—	—	"About 100 ms" reported, hardware not stated

7.7 Screenshots and Visualizations

The figures below are screenshots taken from the project presentation and show the implemented application.



Figure 7.2: Detection, tracking and direction/depth labels on live-camera frames of indoor scenes

Figure 7.2 shows boxes with class labels and identifiers on camera frames, demonstrating detection and tracking on indoor scenes.



Figure 7.3: Image-upload test – input image (left) and annotated output with narration (right)

In Figure 7.3 the output frame shows labels such as “PERSON 77% | NEW | CENTER | FAR” and the narration “A person is visible on the left and appears far. A person is visible near the center and appears far.” This demonstrates the combined detection, direction and relative-depth output for a single image.

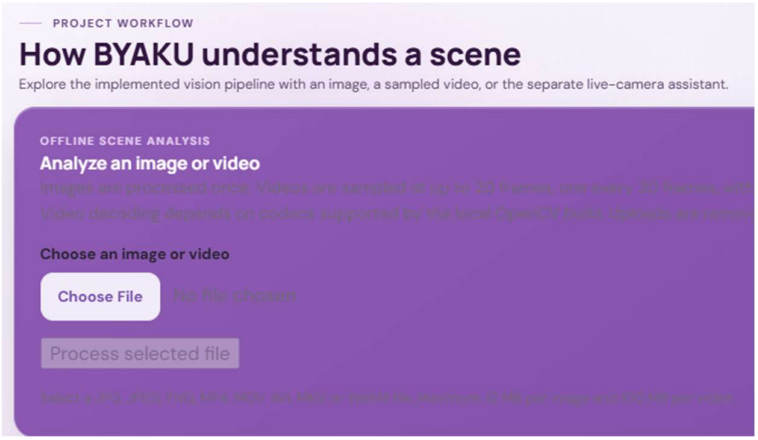


Figure 7.4: Offline video analysis – upload interface and selected video

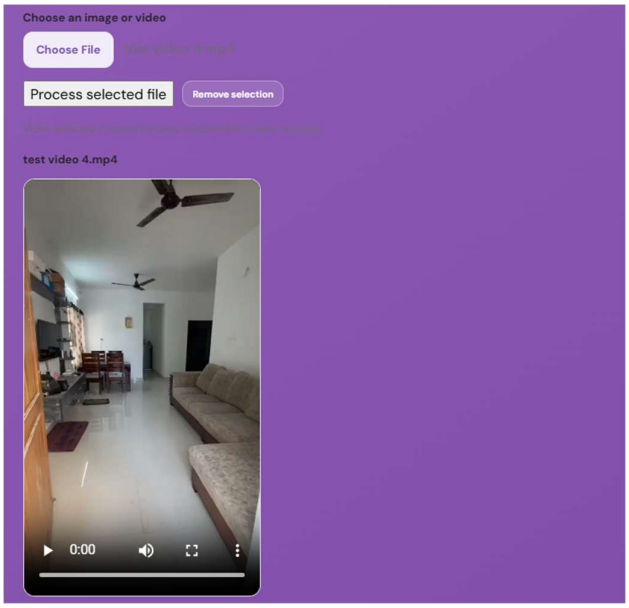


Figure 7.5: Offline video analysis – per-frame result cards for sampled frames

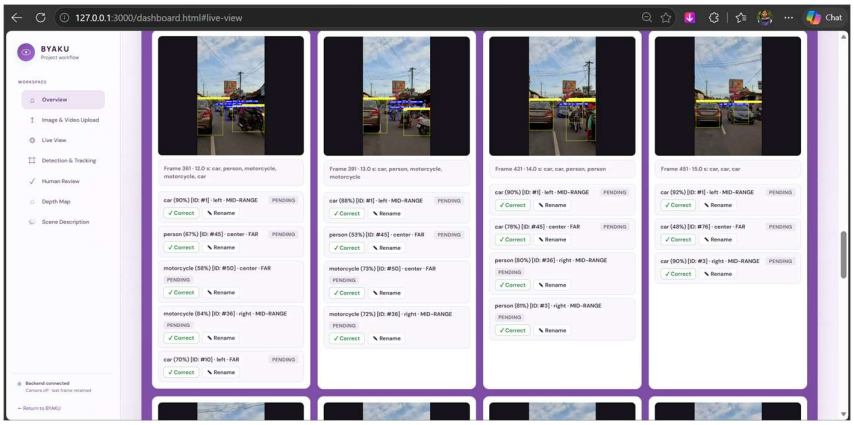


Figure 7.6: Human Review dashboard with per-object confirmation and class correction

Figure 7.6 shows the Human Review panel, where each detected object can be confirmed or corrected and the history is stored with undo support.

7.8 Limitations of the Results

- The test set has only 60 images and 210 boxes, so the metrics have wide uncertainty.
- Only seven classes are covered; the dataset is imbalanced (Figure 5.2).
- No per-class results, confusion matrix, training curves or baseline are available.
- Hardware for the latency claim is not stated.
- Relative-depth zones and direction labels have no quantitative evaluation.
- No user study with blind or low-vision participants is reported, so usefulness for the target users is not experimentally established.

- Test counts (UI 38; backend 411) are reported from an earlier run and were not re-executed.

Experiments needed to complete the analysis: re-run evaluation to export per-class metrics and confusion matrix; evaluate the COCO baseline on the same split; measure latency on stated hardware; evaluate depth-zone correctness on a labelled sample; and, ideally, test with target users.

CHAPTER 8: APPLICATIONS, ADVANTAGES, AND LIMITATIONS

8.1 Real-World Applications

- Describing indoor surroundings for blind and low-vision users (finding a bottle, cup, laptop or phone).
- Assisting caregivers or reviewers who verify what the system reports.
- Offline analysis of recorded videos to review what was detected in a space.
- A teaching example of integrating several deep learning models into a service with human oversight.

These are intended applications; the system is not presented as a certified navigation or safety aid.

8.2 Advantages

- Combines object, direction and relative depth in one spoken sentence.
- Persistent tracking and scene memory reduce repeated or flickering announcements.
- Voice-controlled, screen-reader-aware interface.
- Audited review with undo, without automatic model changes.
- Runs through a local backend and browser, with live and offline workflows separated.

8.3 Technical Limitations

- Detector limited to seven classes; fine-tuned on a small dataset (600 images).
- Depth is relative and coarse (three zones); no distances in metres.
- Narration and query logic depend on detection quality; wrong or missed detections lead to wrong speech.
- Server-side TTS is unavailable and speech playback cannot be confirmed by the backend.
- Uploaded videos are analysed on sampled frames only.
- Feedback does not improve the model automatically.
- Speech interaction is not yet multilingual (listed as future scope).

8.4 Computational Requirements

Detection is reported at about 100 ms per frame, but the hardware is not stated. YOLO11n is a small model, whereas Depth Anything V2 and PyTorch add memory and compute cost; GPU necessity, memory use and load times were not measured or supplied. The initial model load is timed separately by the backend (`initialization_ms`).

8.5 Ethical, Privacy, and Security Considerations

- **Privacy:** camera capture starts only after the user chooses to start; review history does not store frame images, and cached frames expire after 180 s. Camera frames of bystanders may still be processed and should be handled responsibly.
- **Reliability and safety:** the system can be wrong; it should be used as an aid, not as the sole basis for movement decisions.
- **Bias:** the dataset is small and limited to certain classes and scene types; performance in other environments is unknown.
- **Data integrity:** automatic retraining from feedback is deliberately avoided to reduce data poisoning risk.
- **Licensing:** dataset licence (CC BY 2.0) is stated as verified; model and library licences should be checked before redistribution.

CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

BYAKU addresses the difficulty blind and low-vision users have in knowing which objects are around them and where they are. It chains a fine-tuned YOLO11n detector, ByteTrack tracking, Depth Anything V2 relative depth, direction reasoning, scene memory and speech narration, and adds an audited human review process. The repository documents a working backend, browser interface and APIs, and the presentation reports a test-set precision of 85.71%, recall of 82.34%, mAP@0.5 of 75.35% and mAP@0.5:0.95 of 57.58% on 60 images.

Table 9.1 summarises the extent to which each objective is supported by the available evidence.

Table 9.1: Objective-wise achievement status

Objective	Evidence	Status
Detect objects from camera, image, video	Slides 8–12, 27; README	Implemented; detector metrics reported
Track with persistent IDs	Slides 11–12	Implemented; no tracking metrics
Relative depth (near/mid/far)	Slide 15; README	Implemented (optional model); not quantitatively evaluated
Direction (left/center/right, top/bottom)	Slides 12, 15	Implemented; not quantitatively evaluated
Speech narration and voice questions	Slides 16–17; API contract	Implemented; playback not confirmable by backend
Human-in-the-loop review	Slides 22–24; `feedback_store.py`	Implemented; stores feedback only
Accessible, eyes-free interface	Slide 26	Features described; no user study

The objectives are therefore supported at implementation level; quantitative validation exists only for the detector, and no user study has been performed.

9.2 Future Enhancements

Planned work stated in the presentation:

- Improve detection accuracy and speed.
- Evaluate on diverse indoor and outdoor scenes.
- Make depth categories more robust.
- Support speech interaction in different languages.

- Use reviewed annotations to curate datasets and improve the model only through a separately validated training process.

Additional suggestions (not implemented): evaluate a baseline and larger YOLO11 variants, add per-class analysis and a user study, add server-side or confirmed audio playback, measure latency on defined hardware, and add explainability views for detections.

REFERENCES

[1] Ultralytics, “YOLO11,” Ultralytics Documentation, 2024. [Online]. Available: https://docs.ultralytics.com/models/yolo11/

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[3] Y. Zhang et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in Proc. European Conf. Computer Vision (ECCV), 2022. arXiv:2110.06864.

[4] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, “Depth Anything V2,” arXiv:2406.09414, 2024.

[5] A. Kuznetsova et al., “The Open Images Dataset V4,” Int. J. Computer Vision, vol. 128, pp. 1956–1981, 2020, doi: 10.1007/s11263-020-01316-z. (Open Images V7 is a later release of this dataset.)

[6] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Proc. ECCV, 2014, pp. 740–755.

[7] A. Hagberg, D. Schult, and P. Swart, “Exploring Network Structure, Dynamics, and Function using NetworkX,” in Proc. 7th Python in Science Conf. (SciPy), 2008, pp. 11–15.

[8] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” Nature, vol. 521, pp. 436–444, 2015.

[9] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[10] S. Ramírez, “FastAPI,” FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/

[11] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.1.” [Online]. Available: https://www.w3.org/TR/WCAG21/

[12] 24ADI006-DL-Project, “Byaku” (project repository). [Online]. Available: https://github.com/24ADI006-DL-Project/Byaku

Citation details should be re-checked against the publisher pages before final submission.

APPENDIX A: IMPLEMENTATION EVIDENCE AND REPOSITORY STRUCTURE

A.1 Compact repository tree

```
Byaku/
├── Backend/   (run_byaku.py, fastapi_application.py, browser_message_adapter.py,
│              feedback_store.py, frame_timing_profiler.py,
live_camera_producer.py)
├── Workflow/  (detection/depth adapters, spatial reasoning, pipeline, training
tools, tests/)
├── UI/        (browser front end, server/http_websocket_relay.js)
├── Models/    (yolo11n.pt, custom-training checkpoints/runs)
├── Data/      (datasets, manifests, feedback/review_history.json)
├── Docs/      (API_AND_WEBSOCKET_CONTRACT.md, ...)
├── Config/  ├── test videos/  ├── .vscode/
└── README.md, requirements.txt, pyrightconfig.json, .gitignore
```

A.2 Implementation evidence

Table A.1: Implementation evidence

Claim in report	File / document	Purpose
Application entry point and ports	README; `Backend/run_byaku.py`	Launches FastAPI (8765) and relay (3000)
HTTP and WebSocket interface	`Docs/API_AND_WEBSOCKET_CONTRACT.md`; `Backend/fastapi_application.py`	Endpoint and message definitions
Browser message handling	`Backend/browser_message_adapter.py`	WS message adapter
Human-review store, undo, audit	`Backend/feedback_store.py`; `Data/feedback/review_history.json`	Persistent feedback
Timing values	`Backend/frame_timing_profiler.py`	initialization/processing timing
Optional server camera	`Backend/live_camera_producer.py`	Python camera producer
Active detector	README; `Models/yolo11n.pt`	YOLO11n checkpoint
Dependencies	`requirements.txt`	Package list
Tests	`Workflow/tests`	pytest suite (counts from presentation)
Dataset and metrics	Presentation slides 7 and 27	Reported only; raw files not inspected

APPENDIX B: COURSE OUTCOME EVIDENCE

- **CO2 (direct):** detection/tracking/depth pipeline; evidence in Chapter 5, Table 7.1 and Figures 7.2–7.5.
- **CO1 (partial):** add training configuration, loss curves and regularisation details (Appendix B) to make this direct.
- **CO5 (indirect):** transformer-based depth model and query-answering layer; strengthen by documenting how the narration/query module works.
- **CO3, CO4:** no supporting component; not claimed.